

Product Support Agentic AI Agent with Contextual Memory and Multi-Document Reasoning

Software Requirements Specification (SRS)

Title Page

Field	Value
Document Title	Product Support Agentic AI Agent with Contextual Memory and Multi-Document Reasoning - SRS
Document ID	HCL-PSAAI-SRS-001
Version	1.1
Date	2026-08-19
Author	Codex Documentation Review
Company	HCL
Source Document	Product_Support_Agentic_AI_Enterprise_Documentation.docx

Revision History

Version No	Date	Prepared by / Modified by	Significant Changes
1.0	2026-06-13	Codex	Initial enterprise SRS for product support agentic AI solution
1.1	2026-08-19	Codex Documentation Review	Reviewed and aligned the SRS to the current project implementation and repository structure

Table of Contents

- 1. Introduction
- 2. Scope and Overall Description
- 3. Functional Requirements
- 4. Non-Functional Requirements
- 5. External Interfaces and Data Requirements
- 6. Security Requirements
- 7. Acceptance Criteria
- 8. Traceability
- 9. Documentation Consistency Notes
- 10. References

1. Introduction

1.1 Purpose

This document defines the current software requirements baseline for the **Product Support Agentic AI Agent with Contextual Memory and Multi-Document Reasoning** as implemented in the repository. It focuses on externally observable system behavior and constraints rather than internal class design.

1.2 Intended Audience

This SRS is intended for:

- project reviewers
- architects
- developers
- QA/test engineers
- internship evaluators
- operations and support stakeholders

1.3 Product Summary

The system is a document-grounded support assistant that:

- accepts **PDF**, **TXT**, and **CSV** files
- stores uploaded files locally
- extracts text and structured rows
- chunks content and generates local embeddings
- persists a local FAISS knowledge base
- retrieves relevant evidence for user questions
- rewrites ambiguous follow-up questions using recent conversation history
- generates grounded answers through a configurable LLM provider
- returns deterministic citations from retrieval metadata

2. Scope and Overall Description

2.1 In Scope

The current project supports:

- local knowledge-base creation from supported files
- checksum-based duplicate detection during upload
- persistent FAISS indexing with persisted metadata
- semantic retrieval from the persistent knowledge base
- grounded RAG answering using retrieved evidence only
- session-scoped conversation memory
- contextual follow-up query rewriting
- multi-document reasoning when multiple retrieved documents are relevant
- Streamlit-based document ingestion and Q&A workflows
- local knowledge-base clearing and conversation clearing
- developer-facing retrieval/debug visibility in the UI

2.2 Out of Scope

The current repository does not implement:

- web search
- external ticketing integrations
- multi-tenant SaaS deployment
- OCR for image-only documents
- long-term database-backed memory
- agent-tool autonomy beyond the existing LangGraph workflow
- approval workflows

2.3 User Classes

User Class	Description
Knowledge Administrator	Uploads documents, builds the knowledge base, and clears stored knowledge when needed
Support User	Asks questions against indexed documents and reviews grounded answers with citations
Developer / Reviewer	Uses developer tools to inspect retrieval results, debug metadata, and verify pipeline behavior

2.4 Assumptions and Dependencies

ID	Type	Statement
AD-01	Dependency	Python runtime and project dependencies are installed locally
AD-02	Dependency	Local filesystem access is available for <code>data/uploads</code> , <code>data/faiss</code> , <code>data/metadata</code> , <code>logs</code> , and <code>prompts</code>
AD-03	Dependency	The Sentence Transformers embedding model is available locally or can be loaded in the runtime environment
AD-04	Dependency	An active LLM provider is configured through environment variables
AD-05	Assumption	Uploaded files are text-readable or contain extractable text
AD-06	Assumption	The application runs as a single-user or low-concurrency internal workstation application

3. Functional Requirements

Req ID	Title	Requirement	Fit Criterion
FR-001	Supported Upload Types	The system shall accept document uploads in PDF , TXT , and CSV formats only.	Unsupported file extensions are rejected before ingestion.
FR-002	Upload Validation	The system shall validate file extension, file emptiness, and maximum file size before saving uploads.	Invalid uploads are rejected and not saved.
FR-003	Duplicate Detection	The system shall calculate a SHA-256 checksum from uploaded content before saving and shall skip re-ingestion of already indexed content.	Same-content uploads are reported as duplicates and do not add files, chunks, or vectors.
FR-004	Local Upload Persistence	The system shall persist accepted uploads in the configured upload directory and track them in a persistent upload manifest.	Saved documents appear in the upload manifest and remain available after restart.
FR-005	Document Extraction	The system shall extract text from PDF , TXT , and CSV documents using file-type-specific loaders.	PDFs yield page-level extracted documents, TXTs yield document-level text, and CSVs yield row-level extracted documents.
FR-006	Metadata Preservation	The system shall preserve source metadata including file name, document type, page number or row number when available, chunk number, chunk ID, and source path for internal use.	Retrieved results and citations include page/row and chunk identifiers when present.
FR-007	Chunk Generation	The system shall split extracted content into metadata-rich chunks using configurable chunk size and chunk overlap.	Chunk records are created with stable chunk IDs and persisted metadata.
FR-008	Local Embeddings	The system shall generate local embeddings using the configured Sentence Transformers model.	Embeddings are created without calling a remote embedding API.
FR-009	Persistent Vector Storage	The system shall persist the vector index and vector metadata locally.	The FAISS index and metadata files are reusable after restart.
FR-010	Semantic Retrieval	The system shall validate the query, embed it locally, retrieve candidate chunks from the persistent FAISS index, and return top-ranked results.	Retrieval returns ranked chunks with similarity and metadata.
FR-011	Retrieval Quality Controls	The system shall support retrieval reranking, duplicate-result suppression, and adjacent chunk expansion to improve evidence quality.	Returned retrieval results may include rerank metadata and adjacent contextual chunks.

Req ID	Title	Requirement	Fit Criterion
FR-012	Contextual Query Rewriting	The system shall rewrite ambiguous follow-up questions into standalone retrieval queries using recent session history.	When history is available, the retriever may receive a resolved query different from the original user question.
FR-013	Session Conversation Memory	The system shall preserve recent user messages and grounded assistant responses for the active session only.	Follow-up questions can use session context without requiring persistent long-term storage.
FR-014	Grounded Answer Generation	The system shall generate answers using only retrieved knowledge-base context.	The answer-generation prompt and control flow prevent non-grounded answering when evidence is unavailable.
FR-015	Fallback Behavior	The system shall not generate unsupported answers when retrieval is empty or insufficient.	The system returns the configured insufficient-evidence fallback instead of a hallucinated answer.
FR-016	Deterministic Citations	The system shall construct citations from retrieval metadata rather than relying on model-generated references.	Citations include actual file and chunk metadata from retrieval results.
FR-017	Multi-Document Reasoning	The system shall support a multi-document reasoning path when multiple retrieved source documents are relevant and the query indicates comparison or synthesis.	The response may include document evidence summaries and a synthesis result grounded in multiple documents.
FR-018	Knowledge Base Management	The system shall allow users to clear the stored knowledge base and clear conversation history from the UI.	Stored uploads, vector files, metadata, and session conversation data can be reset without changing source code.
FR-019	Developer Diagnostics	The system shall expose developer-facing retrieval, metadata, and pipeline details in the UI without interrupting the main conversation flow.	Developer tools display retrieval details, sources, metadata, and debug information separately from the chat transcript.
FR-020	Logging	The system shall log upload, extraction, chunking, embedding, indexing, retrieval, generation, and clearing events using the application logging framework.	Log entries are written during each major pipeline step.

4. Non-Functional Requirements

Req ID	Category	Requirement
NFR-001	Grounding Safety	The system shall prefer no answer over an unsupported answer when retrieved evidence is insufficient.
NFR-002	Local Data Handling	Uploaded source files, embeddings, vector indexes, and manifests shall remain on the local filesystem.
NFR-003	Configurability	Runtime settings shall be loaded from environment variables with code defaults as fallbacks.
NFR-004	Observability	The application shall provide operational logging and UI-visible debug information for reviewers and developers.
NFR-005	Provider Flexibility	The application shall support a configurable LLM provider abstraction.
NFR-006	Performance	Formal benchmark targets are not fully specified in the current repository. Retrieval and indexing performance are implementation-driven rather than contractually benchmarked in code.
NFR-007	Availability	If the LLM provider is unavailable, the system shall fail safely and report that grounded generation is unavailable instead of inventing an answer.
NFR-008	Portability	The project shall remain runnable as a local Python application using the documented dependencies.
NFR-009	Session Isolation	Conversation memory shall be session-scoped and shall not be persisted as a long-term database.

5. External Interfaces and Data Requirements

5.1 User Interface

The system provides a Streamlit web application with:

- a primary **Ask AI** experience
- an **Ingestion** experience for uploads and knowledge-base management
- a sidebar showing workspace status, provider status, search depth, and developer tools

5.2 LLM Provider Interface

The current code supports:

- **ollama** through the official Ollama Python client
- **gemini** through the official Google Gemini SDK

The active project environment currently selects:

- **LLM_PROVIDER=ollama**
- **OLLAMA_MODEL=qwen3:8b**
- **OLLAMA_BASE_URL=http://localhost:11434**

5.3 Data Storage

The application uses these configured local paths:

Purpose	Path
Uploaded files	<code>data/uploads</code>
FAISS index files	<code>data/faiss</code>
Upload manifest and metadata	<code>data/metadata</code>
Prompt files	<code>prompts</code>
Logs	<code>logs/product_support_agent.log</code>

The persisted vector artifacts are:

- `data/faiss/index.faiss`
- `data/faiss/index.pkl`

The upload manifest is:

- `data/metadata/uploaded_files.json`

6. Security Requirements

Req ID	Requirement
SEC-001	The system shall not expose API keys in the UI, logs, or source-controlled documentation.
SEC-002	The system shall keep uploaded documents and vector data on the local filesystem.
SEC-003	The system shall send only constructed prompt content to the configured LLM provider; raw filesystem paths are not included in model context.
SEC-004	The system shall maintain checksum-based duplicate detection so the same content is not silently re-indexed multiple times.

7. Acceptance Criteria

The current implementation satisfies this SRS when the following are true:

1. A supported document can be uploaded, validated, and stored locally.
2. Duplicate uploads with identical content are skipped without creating new vectors.
3. A knowledge base can be built and re-used after restart.
4. A direct question returns a grounded answer with deterministic citations when relevant evidence exists.
5. A follow-up question can be contextualized using recent session history.
6. A question with insufficient evidence returns the configured fallback message.

7. Clearing the knowledge base removes persisted uploads, vector artifacts, metadata, and current-session conversation state.

8. Traceability

Capability	Traceability
Upload, validation, duplicate detection	FR-001 to FR-004
Extraction and metadata preservation	FR-005, FR-006
Chunking, embeddings, FAISS persistence	FR-007 to FR-009
Retrieval and quality controls	FR-010, FR-011
Query rewriting and memory	FR-012, FR-013
Grounded RAG, fallback, citations	FR-014 to FR-016
Multi-document reasoning	FR-017
Knowledge-base clearing and diagnostics	FR-018, FR-019
Logging and supportability	FR-020, NFR-004

9. Documentation Consistency Notes

- This reviewed SRS now aligns with the current project implementation rather than the earlier Gemini-only and YAML-configuration draft.
- The project supports both Gemini and Ollama in code, but the active local environment is configured for `ollama` with `qwen3:8b`.
- End-user runtime control of `Top-P`, `Top-K`, or prompt editing is **not** implemented in the current UI and is therefore not specified here as a current requirement.
- Formal performance benchmarks beyond what can be inferred from the codebase are left unspecified.

10. References

1. [README.md](file:///D:/hcl/README.md)
2. [app.py](file:///D:/hcl/app.py)
3. [config.py](file:///D:/hcl/src/product_support_agent/config.py)
4. [rag_service.py](file:///D:/hcl/src/product_support_agent/services/rag_service.py)
5. [retriever.py](file:///D:/hcl/src/product_support_agent/services/retriever.py)
6. [query_contextualizer.py](file:///D:/hcl/src/product_support_agent/services/query_contextualizer.py)
7. [memory.py](file:///D:/hcl/src/product_support_agent/services/memory.py)
8. [sections.py](file:///D:/hcl/src/product_support_agent/ui/sections.py)